

ComicCraft: AI Comic Story Creator using Gemini Models

Project Description:

ComicCraft is a web-based application that uses Generative AI to create personalized comic book stories and illustrations from a simple user prompt. Built with FastAPI and integrated with Google's Gemini models along with Stable Diffusion, ComicCraft automatically generates storylines, dialogues, and vivid comic-style imagery. The platform includes an Outline Generator (Gemini Flash), which creates a structured 5-panel comic outline, a Story Writer (Gemini Pro), which expands the outline into narration and character dialogue, and an Image Generator (Stable Diffusion), which draws a comic-style illustration for every panel.

The application takes in user information such as story prompt, main character name, setting, tone, and art style, and generates a panel-by-panel storyline with corresponding illustrations. Users can view the comic preview directly on the web interface and also download the full comic in PDF format, structured with both images and narration.

With a clean, interactive HTML interface and efficient image generation powered by Hugging Face's Diffusers library, ComicCraft ensures that each story feels cohesive and visually engaging. It offers both API endpoints and an intuitive frontend, making comic creation accessible even to non-artists and casual users.

Scenarios

Scenario 1: A user enters a prompt describing their comic story, such as "A brave fox exploring an enchanted forest." The user also specifies the main character's name, the setting (e.g., forest), tone (e.g., dramatic), and art style (e.g., anime). The system combines Gemini 1.5 Flash for the outline, Gemini 1.5 Pro for the story, and Stable Diffusion for image creation to produce a visually immersive, personalized comic strip with multiple panels, images, and text.

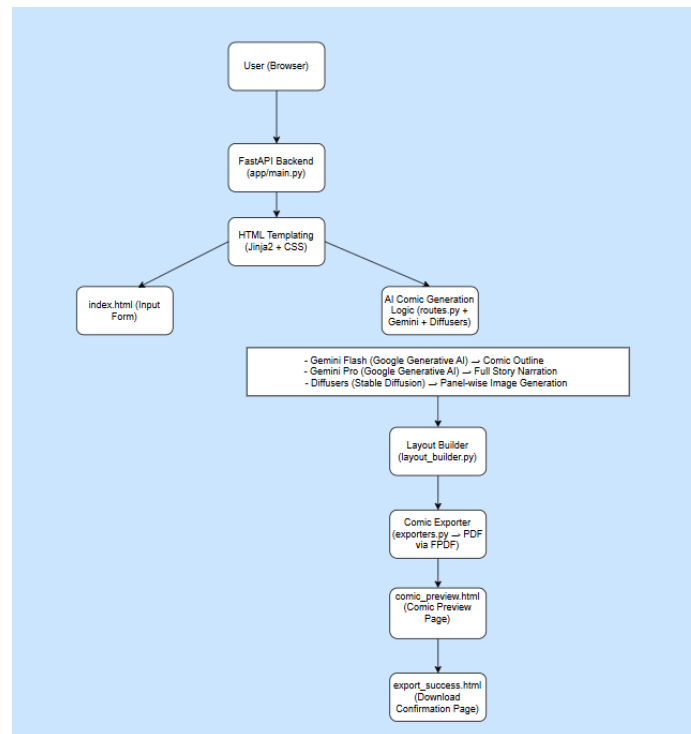
Scenario 2: A user wants a light-hearted, cartoonish feel. On the input form, they select "funny" as the tone and "comic book" as the art style. When they submit, the system regenerates the entire pipeline (outline, story, and images) with prompts tailored to produce a more humorous narrative and a classic comic-book look, letting users iterate until the mood and visual style match their creative vision.

Scenario 3: After reviewing the on-screen preview, the user decides to download the comic. The system then:

- 1 **Layout Binding:** Uses `layout_builder.py` to assemble each panel's title, image, and text into a structured layout.
- 2 **PDF Generation:** Calls `exporters.py` to compile the comic into a PDF file using FPDF, placing images and narrative text onto consecutive pages and saving the PDF with a timestamped filename.

- 3 **Export Success:** Redirects the user to an export success page, confirming the successful download so they can save, print, or share their comic in a professional PDF format.

Technical Architecture:



Description: ComicCraft uses a modular three-tier architecture to turn a short prompt into a complete comic. The frontend (HTML, CSS, Jinja2 templates) captures user inputs and displays the generated comic. The FastAPI backend (app/main.py and routes.py) validates the inputs and orchestrates the workflow. The AI layer calls Gemini Flash for the comic outline, Gemini Pro for the full narration and dialogue, and Stable Diffusion (Hugging Face Diffusers) for panel-wise image generation. layout_builder.py then organizes images and text into a layout, exporters.py compiles the PDF using FPDF, and the app is run locally using Uvicorn.

(Note: ComicCraft does not use a database, so no ER Diagram is required. Generated images and PDFs are stored as files in the static/panels and static/exports folders.)

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Uvicorn ASGI Server:** [Uvicorn Docs](#)
- **Python Programming Proficiency:** [Python Official Docs](#)
- **HTML, CSS, and Jinja2 Template Skills:** [W3Schools HTML/CSS/Jinja2 Tutorials](#)
- **Google Gemini API Familiarity:** [Google Generative AI Documentation](#)
- **Hugging Face Diffusers & Stable Diffusion:** [Hugging Face Diffusers Documentation](#)

- **PyTorch Basics:** [PyTorch Documentation](#)
- **PDF Generation with FPDF:** [FPDF Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Python Package & Environment Setup (pip, virtualenv, or conda):** [Virtualenv Guide](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Research and select the appropriate generative AI models (Gemini Flash, Gemini Pro, Stable Diffusion).
- Activity 1.2: Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- Activity 1.3: Set up the development environment, installing necessary libraries and dependencies.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the core functionalities: outline generation, story generation, image generation, layout building, and PDF export.
- Activity 2.2: Implement the FastAPI backend to manage routing and user input processing.

Milestone 3: routes.py Development

- Activity 3.1: Write the main application logic in routes.py, establishing routes for each feature and integrating AI workflows.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the user interface using HTML and CSS.
- Activity 4.2: Create dynamic templates with FastAPI's Jinja2Templates to display results based on user interactions.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for local deployment by configuring the environment and installing all dependencies.
- Activity 5.2: Test and verify the local deployment using Uvicorn.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting appropriate generative AI models for the ComicCraft application. This involves researching the capabilities and performance of multiple AI models to ensure that the chosen models align with the application's objectives of generating comic storylines, character dialogues, scene descriptions, and comic-style illustrations.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements: Review the specific needs of ComicCraft, focusing on the types of content it must generate:

- Structured comic outlines (panel-by-panel)
- Full comic narration and character dialogues
- Visual comic-style illustrations for each panel

The system must deliver creative, engaging, and visually rich comic outputs for users based on their custom story prompts.

Model Evaluation: Research documentation and capabilities of:

- **Google's Gemini Models:** Gemini Flash (optimized for fast structured outputs) and Gemini Pro (optimized for detailed, creative text generation).
- **Hugging Face Diffusers:** Stable Diffusion model (for high-quality AI-generated images based on prompts).

Compare the selected models based on:

- Response quality for creative writing tasks (narration and dialogues)
- Prompt adherence and richness of detail for scene descriptions
- Speed and resolution of image generation
- Model stability, available documentation, and community support

Practical testing was conducted using different prompts to assess each model's storytelling abilities and image generation performance.

Select the Optimal Models: Choose the AI models that best match ComicCraft's needs:

- **Gemini Flash (models/gemini-1.5-flash):** Selected for fast and structured generation of comic panel outlines based on story prompts.
- **Gemini Pro (models/gemini-1.5-pro):** Selected for detailed, creative comic narration and engaging character dialogues.
- **Stable Diffusion (runwayml/stable-diffusion-v1-5):** Selected for generating high-quality comic-style illustrations from image prompts.

This combination ensures that ComicCraft can deliver rich, creative storytelling alongside vivid, visually appealing comic panels in an automated, efficient manner.

Activity 1.2: Define the Architecture of the Application

The architecture of ComicCraft is structured into three primary components:

- **Frontend** (HTML, CSS, Jinja2)
- **Backend** (FastAPI application)
- **AI Integration** (Google Gemini APIs and Hugging Face Diffusers)

Frontend Responsibilities:

- Users interact with a simple and user-friendly web interface developed using HTML, CSS, and Jinja2 templates.
- Key user input fields: **Story Prompt** (main idea for the comic), **Character Name** (hero of the comic), **Setting** (e.g., forest, school, city), **Story Tone** (e.g., dramatic, funny) and **Art Style** (e.g., anime, realistic).
- Upon form submission, data is sent to the FastAPI backend via a POST request for processing.
- Templates used: index.html (user input), comic_preview.html (display the generated comic), export_success.html (success message after comic export).

Backend Responsibilities:

- FastAPI manages server-side operations and route handling.
- Receives form data from the frontend or JSON API calls, calls the AI models to generate panel outlines, story narration, and illustrations, organizes the complete comic layout, and exports the comic to a downloadable PDF.
- Routes defined in routes.py handle form-based and JSON-based comic generation, image testing, and comic export confirmation.

AI Integration Points:

- **Gemini Flash** is called to generate the structured 5-panel comic outline based on the user's story prompt.
- **Gemini Pro** is called next to create detailed narration and character dialogue from the outline.
- **Stable Diffusion** generates comic-style illustrations from the AI-generated image prompts for each panel.
- The backend receives the AI responses, formats the text and images, and passes them to the frontend templates for preview and export.

All AI model interactions happen dynamically during comic generation, ensuring that every comic is unique and customized based on user input.

Activity 1.3: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python is installed on your system along with pip, which is used to manage the project dependencies.
- **Create a Virtual Environment:** This keeps project-specific packages from conflicting with system-wide installations:

```
D:\projects>python -m venv comiccraft-env
```

```
D:\projects>comiccraft-env\Scripts\activate      # For Windows
```

- **Install Required Libraries:**

```
(comiccraft-env) D:\projects>pip install fastapi uvicorn jinja2 python-multipart gunicorn
```

- **Run Server:** Visit <http://127.0.0.1:8000> for the app and /docs for API testing.

```
(comiccraft-env) D:\projects\comiccraft>uvicorn app.main:app --reload
```

- **Set Up Project Structure:** Create the initial project directory structure for ComicCraft:

```
comiccraft/
|
├─ app/
|   ├── main.py
|   ├── routes.py
|   ├── gemini_flash.py
|   ├── gemini_pro.py
|   ├── image_generator.py
|   ├── layout_builder.py
|   └── exporters.py
|
├─ templates/
|   ├── index.html
|   ├── comic_preview.html
|   └── export_success.html
|
├─ static/
|   ├── panels/      # Generated panel images
|   ├── exports/     # Exported comic PDFs
|   └── fonts/       # Fonts for PDF generation
|
├─ .env              # Environment variables (API keys)
└─ requirements.txt  # Python dependencies List
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Generate Structured Comic Panel Outline

Function: generate_outline() **File:** gemini_flash.py

- Uses the Gemini Flash model to generate a structured 5-panel comic outline from the user's story prompt.
- Each panel contains a panel number, title, scene description, and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.

```
def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini.

    Args:
        user_prompt (str): The user's comic idea prompt.

    Returns:
        list: A list of dictionaries, one for each panel.
    """
    prompt = f"""
    You are a professional AI comic planner.

    Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based on the story idea below:

    STORY: "{user_prompt}"

    Each JSON object must include:
    - "panel" (integer)
    - "title" (string)
    - "scene_description" (string)
    - "image_prompt" (string)

    Respond ONLY in this valid JSON format, without any explanations or markdown:
    [
        {{
            "panel": 1,
            "title": "Title here",
            "scene_description": "Scene description here",
            "image_prompt": "Image prompt for Stable Diffusion"
        }},
        ...
    ]
    """
```

```

try:
    response = model.generate_content(prompt)
    output_text = response.text.strip()

    print("\n 🟡 RAW GEMINI RESPONSE 🟡 \n", output_text)

    # Remove any markdown formatting if present
    if output_text.startswith("```json"):
        output_text = output_text.replace("```json", "").replace("```", "").strip()

    panel_data = json.loads(output_text)

    # Additional structure validation
    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(key in panel for key in ("panel", "title", "scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format or missing keys: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    print("❌ JSON Decode Error:", e)
    print("❌ Full Text Received:\n", output_text)
    return [{"error": f"JSON parsing failed: {str(e)}"}]

except Exception as e:
    print("❌ Unexpected Error:", e)
    return [{"error": f"Generation failed: {str(e)}"}]

```

Generate Comic Story Narration and Dialogue

Function: generate_story() **File:** gemini_pro.py

- Uses the Gemini Pro model to expand the panel outlines into a full comic-style story.
- Creates engaging narration and character dialogues for each panel.
- The output is a single formatted text containing all the panels' stories.

```
def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue
    from a list of comic panel outlines using Gemini 1.5 Pro.

    Args:
        outline (list): A list of strings representing each comic panel's idea.

    Returns:
        str: The generated comic story text or an error message.
    """

    # Format the panel outline as a numbered list for clarity
    formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])

    # Construct the prompt
    prompt = f"""
    You're a comic book writer.

    Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

    Panel Outline:
    {formatted_outline}

    Guidelines:
    - Use a fun and engaging tone, like an actual comic book.
    - Include narration and clearly marked character lines.
    - Keep each panel self-contained but part of a cohesive story.
    """

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

Generate Comic Illustrations from Prompts

Function: generate_image() **File:** image_generator.py

- Uses the Stable Diffusion model to create a comic-style image from the provided image prompt.
- Sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory.
- Returns the file path to the saved image.

```
def generate_image(prompt, filename=None):
    if not filename:
        filename = sanitize_filename(prompt)

    image = pipe(prompt).images[0]
    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)
    image.save(path)
    return path
```

Organize Comic Panels into Layout Structure

Function: build_comic_layout() **File:** layout_builder.py

- Organizes the generated images and full comic story into a structured layout.

- Matches each image with its corresponding panel story.
- The output is a list of dictionaries containing the panel number, image path, and text.

```
def build_comic_layout(image_paths, full_story, outline):  
    # Split the full story into individual panel segments  
    story_panels = full_story.split("**Panel")  
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]  
  
    # Build layout panel by panel  
    layout = []  
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):  
        layout.append({  
            "panel": idx,  
            "title": panel_info.get("title", f"Panel {idx}"),  
            "image_path": image,  
            "text": "\n".join(text.strip().splitlines()[1:]).strip(), # remove panel title  
            "scene_description": panel_info.get("scene_description", "")  
        })  
  
    return layout
```

Export Comic Panel and Story into PDF

Function: save_pdf() **File:** exporters.py

- Compiles the full comic into a multi-page PDF file using the FPDF library.
- Each panel's image and narration are placed neatly on separate pages.
- The final PDF is saved into the static/exports folder with a timestamped filename, and the path is returned.

```
def save_pdf(layout):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Add Unicode font
    pdf.add_font('DejaVu', '', FONT_PATH, uni=True)
    pdf.set_font('DejaVu', '', 12)

    for panel in layout:
        image_path = panel['image_path']
        story_text = panel['text']

        pdf.add_page()

        # Panel title (no bold)
        pdf.set_font('DejaVu', '', 14)
        pdf.cell(0, 10, f"Panel {panel['panel']}", ln=True, align="C")
        pdf.set_font('DejaVu', '', 12)

        # Image placement
        y_image = 30
        image_height = 100
        spacing_after_image = 15

        if os.path.exists(image_path):
            pdf.image(image_path, x=10, y=y_image, w=pdf.w - 20, h=image_height)
        else:
            pdf.set_y(y_image)
            pdf.multi_cell(0, 10, f"Image missing: {image_path}")

        # Text placement below image
        pdf.set_y(y_image + image_height + spacing_after_image)
        story_lines = story_text.strip().splitlines()
        # Remove title line like "***Panel 1: Title**" if present
        if story_lines and story_lines[0].strip().lower().startswith("***panel"):
            story_lines = story_lines[1:]
        cleaned_text = "\n".join(story_lines).strip()
        pdf.multi_cell(0, 10, cleaned_text)

    # Save with timestamp
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI: All routing logic is handled in routes.py. Each route is linked to the corresponding backend function that processes the comic generation workflow developed in Activity 2.1.

Process User Input:

- **HTML Form:** Created in index.html. Users enter the story prompt, main character name, setting, tone, and art style.
- FastAPI captures inputs using Form(...) parameters from the request.
- A Pydantic schema (PromptRequest) validates JSON input for API-based comic generation.

Integrate Gemini API Calls: Each route handler connects directly to the respective AI-powered function modules:

- **/generate** → calls `generate_outline()`, `generate_story()`, `generate_image()` for each panel, `build_comic_layout()`, and `save_pdf()` to export the complete comic.
- **/generate-comic/json** → similar flow as `/generate`, but triggered via a raw JSON API request; returns a JSON response with layout data and the PDF path for API clients.
- **/test-image** → calls `generate_image()` directly with a custom prompt to test image generation separately.

All AI-generated responses (text and images) are processed, cleaned, and injected into the Jinja2 frontend templates (`comic_preview.html`, `export_success.html`) for a seamless user experience.

Milestone 3: routes.py Development

Milestone 3 focuses on setting up all major FastAPI routes for the app, handling user inputs efficiently, integrating the AI generation workflows, and ensuring smooth interaction between backend processing and frontend templates. This step-by-step approach creates a stable foundation for processing prompts, generating comics, and providing a seamless user experience.

Activity 3.1: Writing the Main Application Logic in routes.py

Define the Core Routes in routes.py: Separate routes were established for each of ComicCraft's core functionalities:

- **/** — Loads the homepage (`index.html`) where users submit their story details.
- **/generate** — Handles form submission, processes the input using AI, generates the comic panels, and returns the comic preview page.
- **/generate-comic/json** — An API route that accepts JSON payloads, triggers comic generation, and returns comic layout data and the generated PDF path.
- **/export-success** — Displays a success confirmation page after the comic is downloaded.
- **/test-image** — A developer utility route to test image generation from a direct prompt without full comic creation.

Each route serves as a distinct entry point for different types of user interaction, through browser forms or direct API requests.

Set Up Route Handlers for Each Feature:

- For `/generate`, user inputs are captured using `Form(...)` parameters for story prompt, character name, setting, story tone, and art style.
- For `/generate-comic/json`, inputs are captured from a JSON body using a `PromptRequest` model with `Pydantic`.
- Form data and JSON payloads are passed into the backend functions (`generate_outline`, `generate_story`, `generate_image`, `build_comic_layout`, `save_pdf`).

- Route handlers manage errors via FastAPI's HTTPException system and use try-except blocks to handle unexpected failures such as API errors or JSON decode errors.

Integrate AI Workflows in Each Route: In /generate and /generate-comic/json:

- generate_outline() is called to create a 5-panel structured story.
- generate_story() is called to write narration and dialogues.
- generate_image() is called iteratively to generate images for each panel.
- build_comic_layout() structures the final panel layout.
- save_pdf() compiles the story and illustrations into an exportable PDF.

Each AI-generated output is processed and dynamically passed to the HTML templates (comic_preview.html) or returned as a JSON response for API clients.

Comic Generation Route:

```

21
22
23 @router.post("/generate", response_class=HTMLResponse)
24 async def generate_comic(
25     request: Request,
26     prompt: str = Form(...),
27     character_name: str = Form(...),
28     setting: str = Form(...),
29     tone: str = Form(...),
30     style: str = Form(...)
31 ):
32     try:
33         # Combine user input into a single full prompt
34         full_prompt = (
35             f"{prompt}\n"
36             f"The main character is {character_name}. "
37             f"The setting is a {setting}. "
38             f"The tone is {tone}. The art style is {style}."
39         )
40
41         # Step 1: Generate panel outline
42         outline = generate_outline(full_prompt)
43
44         if not isinstance(outline, list) or not all("image_prompt" in panel for panel in outline):
45             raise ValueError("Invalid outline structure from Gemini response.")
46
47         # Step 2: Generate story
48         full_story = generate_story(outline)
49
50         # Step 3: Generate images
51         images = [generate_image(panel["image_prompt"]) for panel in outline]
52
53         # Step 4: Build Layout
54         layout = build_comic_layout(images, full_story, outline)
55
56         # Step 5: Export to PDF
57         pdf_path = save_pdf(layout)
58         web_pdf_path = "/" + pdf_path.replace("\\", "/") # Make path safe for web
59
60         return templates.TemplateResponse("comic_preview.html", {
61             "request": request,
62             "layout": layout,
63             "pdf_path": web_pdf_path
64         })
65
66     except Exception as e:
67         traceback.print_exc()
68         raise HTTPException(status_code=500, detail=str(e))
69
  
```

Export Success Route:

```

70
71 @router.get("/export-success", response_class=HTMLResponse)
72 async def export_success(request: Request, pdf_path: str):
73     return templates.TemplateResponse("export_success.html", {
74         "request": request,
75         "pdf_path": pdf_path
76     })
77
78

```

Image Generation Route:

```

120
121 @router.get("/test-image")
122 async def test_image(prompt: str = "A futuristic city at sunset, sci-fi, cinematic, artstation"):
123     try:
124         image_path = generate_image(prompt)
125         return {"message": "Image generated successfully", "path": image_path}
126     except Exception as e:
127         raise HTTPException(status_code=500, detail=str(e))
128

```

Milestone 4: Frontend Development

Milestone 4 focuses on building responsive and clean web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, so users can easily submit inputs, view the generated comic panel by panel, and confirm comic export actions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure: index.html was developed as the main entry point for user input. The form captures the Story Prompt, Main Character Name, Setting, Story Tone, and Art Style.

- Each field is properly labeled and grouped using semantic HTML elements.
- The design uses a scenic background image, simple typography, and an organized form layout to encourage creativity and focus on storytelling.
- Navigation is minimal to maintain focus, with a direct flow from input to comic generation.

Design a Responsive Layout Using CSS:

- Embedded CSS styles are applied directly within each HTML file.
- Layouts use fixed width with centered content to ensure readability across different screen sizes.
- Consistent button styling with hover effects was implemented.
- Input fields and result panels use padding, box shadows, and rounded corners for a clean, modern look; the light color scheme enhances visibility.

Create Separate Pages for Each Core Functionality: Three Jinja2-powered HTML templates were created under the templates folder:

- **index.html:** A user-friendly form where users input the story prompt, character name, setting, tone, and preferred art style. Submits data to the /generate FastAPI route via POST.
- **comic_preview.html:** Displays the AI-generated comic panel by panel. Each panel includes the title, scene illustration (image), scene description (in italics), and captions and narration text, so users can review the full storyline sequentially.
- **export_success.html:** Displays a success message after the user downloads their comic as a PDF and encourages users to return and create a new comic.

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering: FastAPI's Jinja2Templates is used for template rendering:

```
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="templates")
```

Each FastAPI route uses TemplateResponse to dynamically render HTML templates based on the user's interaction:

- `/` route renders `index.html`
- `/generate` route renders `comic_preview.html`
- `/export-success` route renders `export_success.html`

This dynamic binding ensures that the comic content (layout, images, text) updates automatically after user input.

Bind Backend Data to HTML Templates:

- **index.html:** Form elements use `name="..."` attributes that directly map to the backend `Form(...)` parameters in FastAPI, and the form submits via POST to `/generate`.
- **comic_preview.html:** Dynamically displays the panel number, title, generated image, scene description (in italics), and captions and narration text, looping through the layout list using the Jinja2 `{% for panel in layout %}` syntax.
- **export_success.html:** Displays a confirmation message using Jinja2 variables, receives the exported PDF path as context, and provides a button for returning to the homepage.

The frontend of ComicCraft is fully integrated with FastAPI, dynamically renders AI-generated comics, and provides a smooth user flow from prompt input to comic export.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the ComicCraft application on a local system using FastAPI. This involves configuring the server environment, installing all required dependencies, setting up static files and templates, and launching the app locally. This milestone verifies that the complete comic generation workflow, including AI text generation, image creation, layout building, and PDF export, runs smoothly before planning deployment to a cloud platform in the future.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment: Create a virtual environment to isolate dependencies from other Python projects, activate it, and install all libraries listed in `requirements.txt` (FastAPI, Uvicorn, Google Generative AI, Hugging Face Diffusers, FPDF, and others):

```
python -m venv env
# For Windows
env\Scripts\activate
# For macOS/Linux
source env/bin/activate

pip install -r requirements.txt
```

Configure Environment Variables:

- Use environment variables to store sensitive data such as the Gemini API key and Hugging Face API token.
- Add them to a .env file in the project root, which is loaded into the application using python-dotenv. Never commit this file to a public repository (add .env to .gitignore).

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your_huggingface_api_key_here
```

Frontend-Backend Rendering via Jinja2: All user interactions are facilitated via Jinja2-rendered templates:

- index.html: Collects user inputs
- comic_preview.html: Displays the generated comic panel by panel
- export_success.html: Confirms that the comic PDF was exported successfully

Activity 5.2: Testing and Verifying Local Deployment

Start the FastAPI Server: Navigate to the project root where app/main.py is located and launch the server with hot reload enabled:

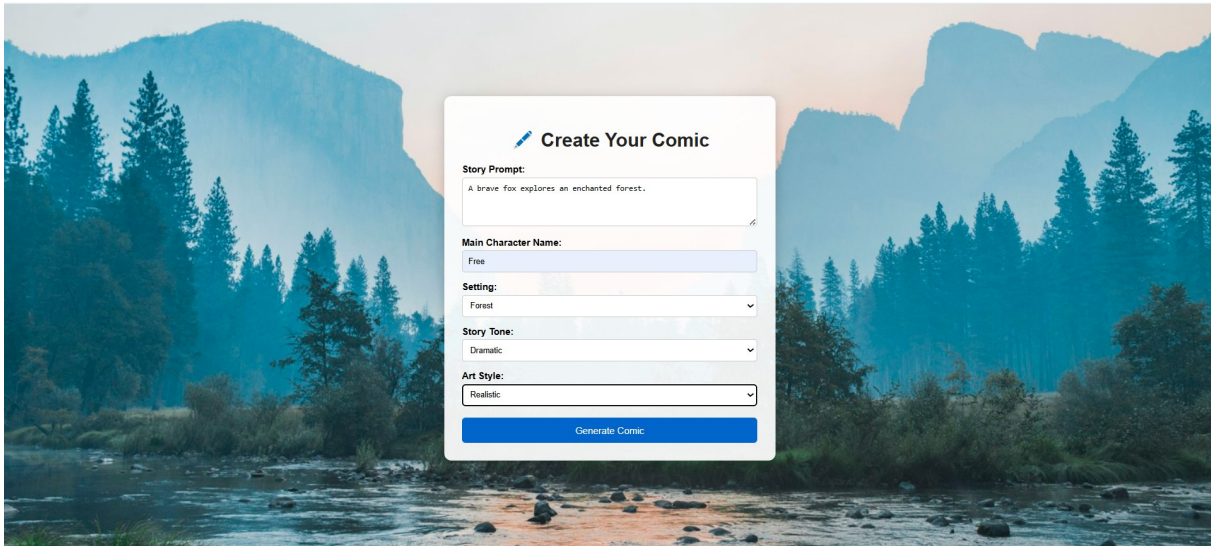
```
(env) D:\projects\comiccraft>uvicorn app.main:app --reload
```

Access the Application Locally: Once the server is running, open a web browser and navigate to <http://127.0.0.1:8000>. This loads the ComicCraft homepage, where users can input their story prompt, character name, setting, tone, and art style to generate a personalized AI-powered comic.

The interactive API documentation provided by FastAPI is available at <http://127.0.0.1:8000/docs>. It lets developers test the available endpoints: /generate-comic/json, /generate, /export-success, and /test-image.

Exploring the Website's Web Pages:

Home Page:



The screenshot shows the homepage of the ComicCraft website. The background is a scenic landscape with a river, forest, and mountains. Overlaid on this is a white form titled "Create Your Comic". The form contains the following fields:

- Story Prompt:** A text input field with the placeholder text "A brave fox explores an enchanted forest."
- Main Character Name:** A text input field with the placeholder text "Free".
- Setting:** A dropdown menu with "Forest" selected.
- Story Tone:** A dropdown menu with "Dramatic" selected.
- Art Style:** A dropdown menu with "Realistic" selected.

At the bottom of the form is a blue button labeled "Generate Comic".

Description: The homepage of ComicCraft is an intuitive starting point for creating AI-generated comic stories, with a clean layout and a scenic background that fosters a creative atmosphere. At the center, a structured form collects the inputs needed for comic generation. Users can enter a Story Prompt describing their comic idea, specify a Main Character Name, choose a Setting (school, forest, space, city), select a Story Tone (light-hearted, dramatic, poetic, funny), and pick an Art Style (anime, pixel art, comic book, realistic). Once the form is submitted, the system uses AI models to automatically generate a complete comic, including storyline outlines, detailed dialogues, illustrations, and an exportable PDF.

Comic Preview – Result Page:

 Your Comic Preview

Panel 1: The Forest's Edge



Free, a red fox with intelligent eyes, stands at the edge of a dark, imposing forest. Sunlight barely penetrates the dense canopy. He looks apprehensive but determined.

Panel 2: Into the Deep Woods



Free cautiously enters the forest, the path barely visible beneath twisted branches and overgrown foliage. An unnatural silence hangs in the air.

****CAPTION:**** Whispers carried on the wind, tales of magic and mystery. Tales of the Whispering Woods...

****NARRATION:**** Free, the bravest fox this side of Silverstream, stood at the threshold. The ancient trees loomed like watchful giants, their shadows a daunting promise.

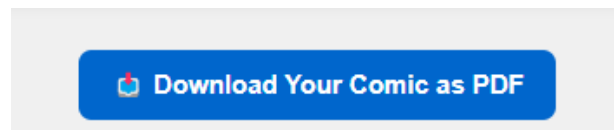
****IMAGE PROMPT:**** Realistic painting of a red fox, Free, standing at the edge of a dark, mysterious forest. Dramatic lighting, sunlight barely penetrating the trees. Focus on the fox's determined expression.

Description: The Comic Preview Page presents the AI-generated comic panels sequentially after the user submits the homepage form. Each panel includes:

- **Panel Title:** Clearly labeled with the panel number and a descriptive title (e.g., “Panel 2: Into the Deep Woods”).
- **Comic Image:** An AI-generated illustration created from the scene’s image prompt.
- **Scene Description:** A short descriptive paragraph, displayed in italics, that sets the atmosphere and context for the panel.
- **Caption and Narration:** The caption gives a brief, ambient description of the environment or background sounds, and the narration details the main character’s actions, emotions, or dialogue within the scene.
- **Image Prompt Reference:** The original artistic description used to generate the illustration, displayed for reference.

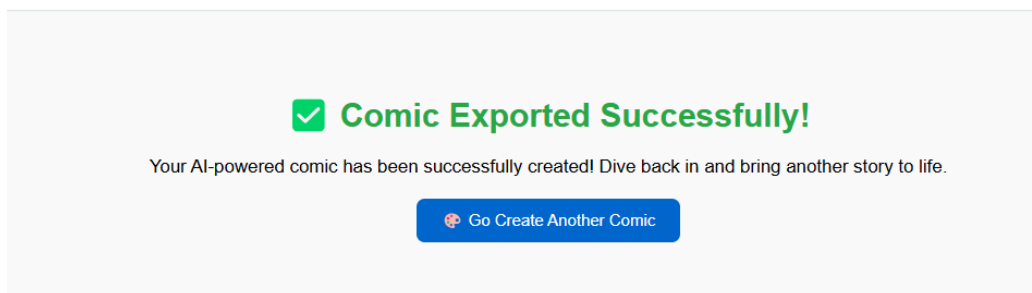
Each panel is clearly separated and visually appealing, making it easy to follow the story’s flow. At this stage, the page focuses purely on comic viewing and story review before any final actions like exporting.

Download Button:



Description: On the Comic Preview Page, a “Download Your Comic as PDF” button is displayed. When users click it, the entire comic, including images, panel titles, scene descriptions, captions, and narration, is compiled into a professionally formatted PDF file and downloaded directly to the user’s device, so they can save, share, and view it offline.

Comic Export Success Page:



Description: After the download is triggered, users are redirected to the Comic Export Success Page, which confirms that their AI-powered comic has been successfully created and downloaded. The page also includes a “Go Create Another Comic” button that encourages users to return to the homepage and start a new comic. This export and success flow provides closure and satisfaction after the creative process while promoting continued engagement with the platform.

Conclusion:

ComicCraft successfully demonstrates the creative potential of AI-driven comic generation. By combining the capabilities of Large Language Models (Gemini Flash and Gemini Pro) for storytelling with Stable Diffusion for comic-style illustration, the application enables users to transform simple prompts into complete, visually engaging comic narratives.

The project provides an end-to-end experience, from personalized story outline generation, full narration, and high-quality image creation to exporting the final comic as a downloadable PDF, all within an intuitive web interface. ComicCraft simplifies comic creation for non-artists, storytellers, and hobbyists, offering a seamless blend of imagination and AI assistance.

While the current version focuses on single comic generation without user account management, the strong modular architecture allows easy future enhancements such as user profiles, comic libraries, per-panel editing, character consistency, progress indicators, cloud deployment, and multi-page story arcs.

Overall, ComicCraft showcases how AI can democratize storytelling and empower users to bring their creative visions to life with minimal effort.